> I'm checking the repository's deploy task and configuration conventions so the `tdps` documentation describes the actual build and Azure upload behavior, rather than relying on a generic Jekyll recipe.

Creating the `tdps` Inner Collection

This repository uses one Git branch per inner collection. The `tdps` branch will contain a complete copy of the site code plus its own collection metadata and configuration. Its generated site will be uploaded beneath `/tdps` in the Azure static website's `$web` container.

The published collection URL will be:

```
https://digitalgrinnell-secondary.z19.web.core.windows.net/tdps/
```

1. Start from Current `main`

Make sure the local `main` branch is current before creating the collection branch:

```
git switch main
git pull --ff-only origin main
git switch -c tdps
```

`git switch -c tdps` creates a new local branch named `tdps` based on the currently checked-out `main` commit, then switches the working directory to it.

Everything you edit after this point, including the root `_config.yml`, belongs to the `tdps` branch unless you later switch branches.

2. Add the Collection Metadata

Create the item-level metadata file:

```
_data/tdps.csv
```

The file name matters because the CollectionBuilder configuration refers to it by its base name: `tdps`, without `.csv`.

The exact metadata columns must fit the site's CollectionBuilder conventions. A practical way to begin is to copy a comparable existing CSV in `_data` and retain required headers, then replace its records with the `tdps` collection's data.

> The example below is actually `NOT sage advice` because the `_data/digital_collections.csv` file has a very different column/header set than most

> content collections.

For example:

```
cp _data/digital_collections.csv _data/tdps.csv
```

Review the copied headers and data carefully before building. Each item should have a unique `objectid`; this identifier is used when Jekyll generates item pages.

3. Configure the Branch-Specific Site

Edit the `_config.yml` located at the repository root:

```
CB-Digital-Grinnell/
├── _config.yml       <-- edit this file on the tdps branch
├── _data/
│   └── tdps.csv
└── ...
```

Find the existing `url`, `baseurl`, and `metadata` settings and set them as follows:

```
url: "https://digitalgrinnell-secondary.z19.web.core.windows.net"
baseurl: "/tdps"
metadata: tdps
```

These are active YAML configuration values, not comments.

`url` is the static website's public domain. It does not identify a Git branch.

`baseurl` is the collection's directory below that domain. Set it to `"/tdps"` because the Azure deployment uploads the generated site into the `tdps/` path within `$web`.

`metadata` selects `_data/tdps.csv`. Do not include the `.csv` extension.

A useful rule is:

```
metadata: tdps       -> _data/tdps.csv
baseurl: "/tdps"     -> https://...web.core.windows.net/tdps/
destination-path tdps -> $web/tdps/
```

Those three values should stay aligned.

4. Build the Collection

Run this while the `tdps` branch is checked out:

```
rake deploy
```

In this repository, `rake deploy` sets `JEKYLL_ENV=production` and executes:

```
bundle exec jekyll build
```

Jekyll automatically reads `_config.yml` from the root of the currently checked-out branch. Therefore, when `tdps` is checked out, the build uses that branch's:

```
baseurl: "/tdps"
metadata: tdps
```

It writes the generated static site to:

```
_site/
```

`rake deploy` builds the site only. It does **not** upload anything to Azure.

Before uploading, inspect the generated result locally if desired:

```
bundle exec jekyll serve
```

Then visit:

```
http://127.0.0.1:4000/tdps/
```

Use `Ctrl-C` to stop the local server.

5. Upload the Generated Site to Azure

The local `.env` file supplies non-secret Azure environment settings such as the storage account name and login authentication mode. It should remain untracked.

Load it, then upload the entire generated `_site` directory into the collection's Azure path:

```
source .env

az storage blob upload-batch \
  --destination '$web' \
```

```
  --destination-path tdps \
  --source _site \
  --overwrite
```

This maps files such as:

```
_site/index.html
_site/assets/css/main.css
_site/items/example.html
```

to:

```
$web/tdps/index.html
$web/tdps/assets/css/main.css
$web/tdps/items/example.html
```

The single quotes around `'$web'` are intentional: they prevent the shell from trying to expand `$web` as an environment variable.

The command assumes Azure CLI authentication is already active. If necessary, authenticate first:

```
az login
```

6. Verify the Published Collection

After the upload finishes, check:

```
https://digitalgrinnell-secondary.z19.web.core.windows.net/tdps/
```

Also open several item pages and inspect the browser console for missing assets, broken links, or incorrect paths. If styles or links incorrectly point to the root site, recheck:

```
baseurl: "/tdps"
```

and rebuild before uploading again.

7. Commit and Publish the Branch

Once the collection is validated, commit its source files and push the branch to GitHub:

```
git status
git add _config.yml _data/tdps.csv
git commit -m "Add TDPS collection"
git push -u origin tdps
```

Add any additional collection-specific pages, images, or configuration files to the `git add` command as needed.

The GitHub branch and the Azure deployment are separate actions:

- `git push -u origin tdps` preserves and shares the collection source code.
- `az storage blob upload-batch ... --destination-path tdps` publishes the generated static site.

Repeat Deployment

For later updates to `tdps`:

```
git switch tdps
rake deploy

source .env
az storage blob upload-batch \
  --destination '$web' \
  --destination-path tdps \
  --source _site \
  --overwrite

git add <changed-files>
git commit -m "Update TDPS collection"
git push
```

Keep `main` as the shared/root-site branch; make collection-specific edits on `tdps`.